Network Backend Roadmap v2

Protocols from RFCs, projects to ship, four specialisation tracks and a 6-month schedule

Target role: backend developer in networking · Languages: Go, Rust, C/C++ · v1 + 30-project ideas list · Prepared 6 November 2026

24	3 + 1	30	26
protocols to implement from scratch (20 + 4 new)	core projects + one specialisation track (of 4)	ideas from the project list, all mapped (Appendix)	weeks of schedule at 8–10 hours a week

- 1. How to use this plan
- 2. Protocols to implement from scratch
- 3. Core projects (one repository each)
- 4. Specialisation tracks (pick one)
- 5. Optional projects
- 6. Schedule at a glance
- 7. Which job requirement you practise where
- 8. Checklist for every repository
- Appendix · Where the 30 ideas went

1. How to use this plan

- **The rule.** Implement each protocol from its RFC (or 3GPP spec), then verify it with real tools (*dig, curl, tcpdump, Wireshark, FRR, SIPp, radtest, Open5GS*). That lets you say “my server answers dig” instead of “I know DNS”.
- **Language order.** Go for protocols, servers and control planes. C for eBPF/XDP and kernel-facing datapaths. Add **Rust** after 3–4 weeks by porting a protocol you already know (TFTP first, then DNS). Treat **C++** as an extra only if your target (Cisco, Ericsson, Motorola, Nokia) asks for it. v1 said Go first; the ideas list assumed C/C++ first; v2 keeps Go first and uses C where the kernel needs it.
- **Pick one direction.** After the core path (weeks 1–13) choose **one** track: 5G user plane, data plane, security or routing. Three polished repositories beat fifteen unfinished ones.
- **Two languages in one project.** Core in one language, tools and tests in another (datapath in C or Rust, control plane and reports in Go or Python). Or write one module twice (for example a parser in C and in Rust) and compare speed, memory bugs found by ASan, and the time it took. Version B does not have to be complete.
- **Quality over quantity.** Every protocol gets tests on real packets, parser fuzzing, a benchmark and a short README. Do not write cryptography from scratch: take TLS, QUIC, SSH, IPsec and VPN handshakes from libraries.

> **What changed from v1.** Protocol numbers 1–20 are unchanged; 21–24 are new (RIPv2, GTP-U, PFCP, Diameter basics). Every protocol now has a **tier** and a time estimate. The 30 ideas became four tracks plus a mapping appendix instead of 30 extra repositories. A 6-month schedule was added. Two deliberate deviations from the ideas list: the WireGuard-style VPN takes its handshake from a library (no hand-designed crypto), and port scanners and honeypots follow the local-only rule for network tests.

2. Protocols to implement from scratch

Legend. The first language in a row is where you start; the others are reasonable alternatives. Python = prototypes, tests and scripts. Node/TS = Node.js and TypeScript. **Tier 1** = core path, everyone does it in weeks 1–13. **Tier 2** = needed by a core project or a track. **Tier 3** = optional or after month 6. Time = planning estimate at 8–10 hours a week (taken from the ideas list where it gave one, otherwise my guess); re-measure after your first two protocols.

Protocol (RFC)	Languages	What to build	Verify with	Useful for
LEVEL 1 · Foundations				
1. TFTP RFC 1350 *Tier 1 · ~1–2 wk*	**Go** · Rust · C++ · Python	UDP client and server: RRQ/WRQ, 512-byte blocks, ACKs, retransmission and timeouts. A good warm-up, and the first protocol you port to Rust.	tftp-hpa, Wireshark	UDP, state machines, timeouts
2. TCP and HTTP/1.1 RFC 9293, 9112 *Tier 1 · ~3 wk*	**Go** (no net/http) · Rust · C++ (epoll)	TCP server with framing and timeouts, then your own HTTP server: request parser, headers and header limits, keep-alive, chunked encoding, slowloris protection, connection limits, a /metrics endpoint, a few routes.	curl, wrk, k6, Wireshark	Proxies, APIs, any backend
3. WebSocket RFC 6455 *Tier 3 · ~1 wk*	**Go** · Rust · Node/TS	Upgrade handshake, frames, masking, ping/pong, close handshake.	websocat, browser	Dashboards, UniFi, notifications
4. DNS RFC 1034, 1035, 6891 (EDNS) *Tier 1 · ~3–4 wk*	**Go**, then port to **Rust** · C++	Message parser and builder with name compression, stub resolver, authoritative server over UDP and TCP (TC bit), TTL cache with negative caching, response rate limiting against amplification. Optional: DNS over TLS (RFC 7858) with crypto/tls.	dig, kdig, Wireshark	Cloudflare, ISPs, project 3
5. DHCP RFC 2131, 2132 *Tier 3 · ~2 wk*	**Go** · Rust · C++	Server with an address pool, leases and options, and a REST API for management (IPAM).	dhclient in a network namespace, Wireshark	ISPs, networking, IPAM
LEVEL 2 · Packets and lower layers				
6. Ethernet, ARP, IPv4, ICMP, UDP, TCP RFC 826, 791, 792, 768, 9293 *Tier 1 · ~2–3 wk*	**Go**, then port to **Rust** · C++ (libpcap)	Raw-byte parser that reads pcap files (write the parsing yourself; use gopacket or libpcap only for capture), flow table by 5-tuple, your own ping and traceroute with raw sockets, ARP spoofing detection. Raw-socket tests only against localhost or network namespaces.	Wireshark, tcpdump	Packet processing, eBPF, IDS, security
7. NTP RFC 5905 *Tier 3 · ~1 wk*	**Go** · Rust · C++	SNTP client and a simple server: offset, delay, modes, stratum.	chronyc, ntpdate -q	Systems, time telemetry
8. STUN RFC 8489 *Tier 3 · ~1 wk*	**Go** · Rust · Node/TS	Client and server: XOR-MAPPED-ADDRESS, NAT type detection.	Wireshark, a STUN server in a container	WebRTC, VoIP
9. TLS 1.3 (ClientHello only) RFC 8446 *Tier 2 · ~1–2 wk*	**Rust** · Go	Handshake parser: SNI, ALPN, versions, cipher suites, extensions; JA3 and JA4 fingerprints and a table of known fingerprints. No cryptography to write.	openssl s_client, curl -v	Security, traffic filtering
10. Reliable transport over UDP inspired by RFC 9293, 9000 *Tier 2 · ~3 wk*	**Rust** · Go · C++	Sliding window, retransmission, congestion control (Reno), simulated packet loss.	tc netem, iperf3	How TCP and QUIC work inside, layer 2

Protocol (RFC)	Languages	What to build	Verify with	Useful for
11. GRE and VXLAN RFC 2784, 7348 *Tier 2 · ~2 wk*	**Rust** · Go · C++	Encapsulation over a TUN device, two hosts in separate network namespaces. Foundation for the tunnel in track C and the EVPN lab in track D.	ip, tcpdump	SD-WAN, VPN, cloud
LEVEL 3 · Routing and telemetry				
12. BGP-4 RFC 4271 (BMP: RFC 7854) *Tier 2 · ~4–5 wk*	**Go** · Rust · C++	State machine, OPEN, KEEPALIVE, UPDATE, NOTIFICATION; ORIGIN, AS_PATH, NEXT_HOP, LOCAL_PREF, MED; route table with best-path selection; install routes in the kernel through netlink. Peer with FRR in containerlab.	FRR (vtysh), Wireshark	Cloudflare, carriers, project 2, track D
13. OSPFv2 (advanced) RFC 2328 *Tier 3 · ~4 wk*	**Go** · Rust	Hello, adjacencies, type-1 LSAs, SPF algorithm.	FRR, Wireshark	Routing, carrier networks
14. SNMP v2c RFC 3416 *Tier 2 · ~2 wk*	**Go** · Python · Rust	Agent with a few OIDs and a manager: GET, GETNEXT, WALK, BER encoding.	snmpwalk, snmpget	Monitoring, ISPs
15. sFlow v5 / IPFIX sFlow: sflow.org spec; IPFIX: RFC 7011 *Tier 2 · ~2–3 wk*	**Go** · Rust	Datagram collector, IPFIX template handling, flow aggregation (top talkers, protocols, AS), storage in ClickHouse.	hsflowd, softflowd	Telemetry, project 2
16. Syslog RFC 5424 *Tier 3 · ~1 wk*	**Go** · Rust	UDP and TCP receiver, parser, index, simple queries.	logger, rsyslog	Observability, Splunk
LEVEL 4 · ISP and telecom				
17. RADIUS RFC 2865, 2866 (accounting), 5176 (CoA) *Tier 2 · ~3 wk*	**Go** · Rust · C++ · Java	Server for Access-Request / Accept / Reject, PAP and CHAP, attributes, Message-Authenticator, accounting, optional CoA/Disconnect. Subscribers in PostgreSQL, traffic accounting into ClickHouse.	radtest, eapol_test, FreeRADIUS as client	ISPs, Wi-Fi, UniFi
18. SIP + SDP RFC 3261, 8866 *Tier 3 · ~4 wk*	**Go** · Rust · Node/TS · C++ (PJSIP)	Parser, REGISTER with digest authentication, INVITE / 200 / ACK / BYE, user location (Redis), call routing.	SIPp, Linphone, Wireshark	VoIP, Ubiquiti Talk, Cisco, Motorola
19. RTP / RTCP RFC 3550 *Tier 3 · ~2–3 wk*	**Go** · Rust · C++	Receiver, jitter buffer, loss and jitter statistics, a simple RTP relay.	SIPp (media), ffmpeg, Wireshark	Media, telecom, WebRTC
20. MQTT 3.1.1 / 5.0 OASIS specification *Tier 3 · ~2 wk*	**Go** · Rust · Node/TS	Broker: CONNECT, PUBLISH, SUBSCRIBE, QoS 0 and 1, retained messages.	mosquitto_pub / mosquitto_sub	IoT, UniFi Connect
LEVEL 5 · New in v2: 5G user plane and extras				
21. RIPv2 RFC 2453 *Tier 2 · ~1–2 wk*	**Go** · Rust · C++	Route table with split horizon, 30 s updates, timeout and garbage-collection timers, multicast updates; static routes first, then dynamic. Runs in network namespaces next to your software router.	FRR (ripd), Wireshark	Routing basics, vendor roles, track D
22. GTP-U 3GPP TS 29.281 *Tier 2 · ~2 wk*	**C** · Go · Rust	G-PDU encapsulation and decapsulation, TEID handling, echo request/response, extension headers; over a TUN device first, XDP later.	Wireshark (GTP dissector), UERANSIM + Open5GS	5G user plane, track A
23. PFCP 3GPP TS 29.244 *Tier 2 · ~3 wk*	**Go** · C++ · Rust	IE (TLV) codec; Association Setup, Heartbeat, Session Establishment / Modification / Deletion; session state machine, retransmissions, timeouts; a mock UPF for tests without a 5G core.	Open5GS SMF/UPF as peers, Wireshark	Nokia, Ericsson, Samsung R&D, track A

Protocol (RFC)	Languages	What to build	Verify with	Useful for
24. Diameter (basics) RFC 6733; Credit-Control: RFC 8506 *Tier 3 · ~2 wk*	**Go** · C++ · Python	AVP codec, CER/CEA, DWR/DWA and a minimal Credit-Control exchange; reuse the subscriber database from RADIUS.	freeDiameter, Wireshark	Operator OSS/BSS, AAA

LEVEL 6 · Use libraries (do not write from scratch)

What	Libraries	Why not from scratch
QUIC and HTTP/3	quic-go (Go), quinn (Rust)	A huge specification; what you learn in item 10 is enough
TLS	crypto/tls (Go), rustls (Rust)	Never write your own cryptography; it is a security risk
SSH and NETCONF	x/crypto/ssh (Go), russh (Rust)	Cryptography. Focus on the client and configuration automation
gRPC and gNMI	grpc-go, tonic (Rust)	It is enough to design the API in protobuf
WireGuard / Noise	wireguard-go, boringtun, a vetted Noise library	Build the tunnel from item 11, the anti-replay window and MTU/MSS handling yourself, but take the handshake and encryption from a library. The ideas list suggested designing a handshake with X25519 + ChaCha20-Poly1305; do not.
IPsec and IKEv2	strongSwan	Cryptography plus a large state machine. Automate it and benchmark it in a lab instead (AES-GCM vs ChaCha20, MTU, failover)
eBPF loaders and maps	libbpf (C), cilium/ebpf (Go), aya (Rust)	Write the XDP/eBPF programs yourself; use a loader library for the plumbing
Multi-pattern matching	aho-corasick (Rust), Hyperscan	For the IDS in track C: write a simple Aho-Corasick first to understand it, then compare with the library

3. Core projects (one repository each)

The three projects turn the protocols above into the systems that job postings ask about. Each has milestones (MVP, v1, v2), so you can show a project after the first one.

Project 1 · edge-proxy: reverse proxy with a control plane	Go · Rust · C++
Goal	Build an L7 proxy that you steer through an API, the way API gateways and CDNs do.
Components	**Data plane:** HTTP/1.1 and HTTP/2 proxy with TLS termination, load balancing (round-robin, least-connections), health checks, rate limiting (token bucket), circuit breaker, timeouts. **Control plane:** REST and gRPC for managing routes, upstreams, certificates and policies. **Data:** configuration in PostgreSQL, rate-limit counters in Redis.
Technology	Prometheus and Grafana, OAuth2 / JWT on the API, Docker, Kubernetes (kind), Helm, GitHub Actions, k6 load tests.
Languages	**Go** (primary, whole system). **Rust** (tokio, hyper, axum): an alternative, or the data plane in v2. **C++** (Boost.Beast, Asio): possible, but more work.
Milestones	**MVP:** HTTP/1.1 proxy with round-robin and health checks, config from a file. **v1:** REST control plane, PostgreSQL, config reload without dropping connections, Redis rate limiting, metrics. **v2:** TLS, HTTP/2, JWT, Helm and Kubernetes, k6 test, outage runbook.
Done when	Throughput and p99 latency are measured (numbers in the README, with the command). Config reload does not interrupt requests. When an upstream goes down, traffic shifts to healthy instances. Tests and parser fuzzing are in place.
New in v2	Reuse the connection limits and slowloris protection from your HTTP server (protocol 2) and load-test with your own traffic generator (track B) next to k6. Scheduled in Phase 2 (months 7–9); start it first if you target Cloudflare or Akamai.
Covers	Cloudflare (TLS/HTTP proxy, APIs, Go), Akamai (API gateway, rate limiting, load balancing, Kubernetes), Ubiquiti (OAuth, Docker, SQL, defensive programming).

Project 2 · netops-platform: network telemetry and configuration	Go · Rust · Python
Goal	Collect data from network devices, store and version their configuration, and alert on failures.
Components	**Lab:** containerlab with 3–4 FRR routers (BGP, OSPF). **Collectors:** SNMP, sFlow, configuration over SSH and NETCONF, BGP monitor (the session from item 12, or BMP). **Processing:** NATS or Kafka queue, aggregator. **Data:** ClickHouse (flows), PostgreSQL (inventory, configuration history with diff and rollback). REST and gRPC API, alert rules.
Technology	Grafana, Prometheus, Docker Compose or Kubernetes, GitHub Actions, integration tests on the lab.
Languages	**Go** (primary). **Rust** (sFlow and SNMP collectors). **Python** (lab scripts, integration tests). **C++** is possible for collectors.
Milestones	**MVP:** device inventory, config fetch over SSH, versioning and diff. **v1:** SNMP, metrics, Grafana dashboard. **v2:** sFlow → NATS → ClickHouse, BGP monitor, alerts.
Done when	Taking a lab link down raises an alert in under 30 s. Configuration history with diff and rollback. A flow chart built from ClickHouse. One command brings up the whole lab.
New in v2	Add the EVPN-VXLAN leaf-spine lab (idea 27: FRR in containerlab, convergence and ECMP tests), gNMI subscriptions next to SNMP, and the QoS lab (idea 29) as a lab scenario. Scheduled in Phase 2 (months 10–12); start it first if you target Cisco, Juniper or Arista.
Covers	Cloudflare (Traffic and Network Platforms teams: ClickHouse, NETCONF, Grafana, PostgreSQL, BGP), Cisco, Arista, Juniper (network automation and telemetry).

Project 3 · dns-xdp: DNS server with in-kernel protection	Rust · Go · C
Goal	Build a DNS server and a Linux kernel packet filter that limits traffic before it reaches the application.
Components	**Server:** authoritative from zone files plus a cache, UDP and TCP, EDNS, response size limit. **Filter:** XDP/eBPF program that rate-limits packets per source address and counts statistics in BPF maps. **Measurements:** traffic generator (dnsperf), packets-per-second chart, metrics in Grafana.
Technology	eBPF/XDP, BPF maps, tokio, Prometheus, Docker, fuzzing (cargo-fuzz), CI.
Languages	**Rust** (primary: tokio plus aya). **Go** (cilium/ebpf). **C** / **C++** with libbpf: closest to the kernel, if you target C/C++ companies.
Milestones	**MVP:** server from zone files, parser ported from Go. **v1:** cache, TCP, EDNS, metrics. **v2:** XDP filter, throughput tests with and without it.
Done when	A packets-per-second chart with and without XDP. The DNS parser survives fuzzing. The server answers dig for test zones. A short write-up of what happens under a query flood.
New in v2	Scheduled in weeks 14–18, right after the Go DNS server. The XDP filter is idea 9: LPM_TRIE blocklist, per-CPU token bucket, SYN-flood counter, ring-buffer events. Add DNS over TLS through rustls and response rate limiting from idea 7. The same filter written in C with libbpf is the comparison for track B.
Covers	Cloudflare (DNS, eBPF/XDP, Rust, systems programming), networking companies that use C/C++ (Motorola, Ericsson).

4. Specialisation tracks (pick one)

Choose **one** track at the checkpoint in week 13 and build it in weeks 19–23. The tracks come from the four portfolio sets in the ideas list. Tracks B and D overlap with core projects 2 and 3 (XDP filter, BGP, telemetry): keep those pieces in the core repositories and do only the rest in the track repository.

Track A · mini-5g-user-plane: a 5G user plane you can swap into a real core	Go · C · Rust (version B)
Goal	Run a 5G core lab and replace its UPF with your own, driven over PFCP. One story: lab, protocol, datapath, tests.
Components	**Lab (idea 15):** Open5GS + UERANSIM (or srsRAN over ZMQ), Docker Compose, pytest end-to-end tests (UE registration, PDU session, ping and iperf through the tunnel), annotated pcaps. **PFCP library and CLI (17, protocol 23):** IE codec, session messages, state machine, mock UPF. **UPF (14, protocol 22):** GTP-U encapsulation and decapsulation over TUN (XDP optional), simplified PDR/FAR/QER/URR, rate limit and usage counters. **Tools:** GTP-U traffic generator and UE simulator (18), signalling analyser from pcap with Mermaid sequence diagrams (16).
Technology	Open5GS, UERANSIM, Docker Compose or Helm, Prometheus, Grafana, Wireshark/tshark, pytest, fuzzing of the PFCP and GTP-U codecs.
Languages	**Go** (PFCP control plane, tools). **C** (datapath, optional XDP). **Python** (lab tests, analyser). Version B: the whole thing in **Rust**.
Milestones	**MVP:** lab starts with one command, pytest passes registration and PDU session, pcap with the call flow described. **v1:** PFCP library and CLI talking to Open5GS, mock UPF, fuzzed codec. **v2:** your UPF carries real UERANSIM traffic, QER and URR work, benchmark against the Open5GS UPF, traffic generator.
Done when	A UE pings through your UPF in place of the Open5GS UPF. The benchmark against the Open5GS UPF has numbers and the exact command. PFCP and GTP-U fuzz targets run 30 s without a crash. The README shows the uplink and downlink packet flow from a pcap.
Covers	Nokia, Ericsson, Samsung R&D and other 5G core vendors.

Track B · packet-fastpath: from parser to XDP balancer, with measurements	C/eBPF · Go · Rust (aya) · C++ DPDK (version B)
Goal	One narrative: parser, filter, balancer, measurement. Each step reuses the one before it.
Components	Sniffer and parsers (idea 2, protocol 6) → traffic generator (13: IMIX, burst, constant bitrate, latency by timestamps) → **XDP firewall** (9: LPM_TRIE blocklist, per-CPU token bucket, SYN-flood detection, ring-buffer events, CLI) → **XDP L4 load balancer** (8: Maglev hashing, LRU_HASH connection table, IPIP or DSR, Go control plane with health checks) → **eBPF observability** (12: kprobes on tcp_retransmit_skb and kfree_skb, RTT histograms, Prometheus). Optional: DPDK forwarder (10).
Technology	libbpf, bpftool, cilium/ebpf or aya, network namespaces or a VM, perf and flamegraphs, tc netem, Prometheus, Grafana.
Languages	**C** (eBPF programs). **Go** (control plane, generator). **Rust** (aya) as the alternative loader. **C++** with DPDK as version B.
Milestones	**MVP:** XDP firewall and a packets-per-second chart against nftables. **v1:** traffic generator and the L4 balancer with a control plane. **v2:** eBPF observability dashboard; DPDK forwarder if time allows.
Done when	A Mpps chart for XDP vs nftables under the same attack, with commands and the kernel version. The balancer keeps serving when a backend dies. Programs pass the verifier on a documented kernel. The README says how to run the loader.
Covers	Cloudflare, Akamai, F5, Cisco, Juniper, Arista.

Track C · security-gateway: tunnel, firewall, IDS and telemetry in one box	Rust · Go · Python · C++ (version B)
Goal	One gateway with a tunnel, a policy firewall, an IDS/IPS and telemetry, tested with attacks and fuzzers.
Components	**Tunnel (5):** TUN, UDP encapsulation, anti-replay window, keepalive, MTU and MSS clamping; handshake and encryption from a library. **Firewall-as-code (25):** YAML policy → nftables, Scapy attack tests in namespaces, policy diff in CI. **IDS/IPS (20):** AF_PACKET capture, NFQUEUE inline mode, TCP stream reassembly, a Suricata-subset rule engine (content, pcre, flow), Aho-Corasick, EVE-JSON alerts to Loki and Grafana. **TLS fingerprinting (21, protocol 9):** ClientHello parser, JA3 and JA4. **Honeypot (22)** and **fuzzing of every parser (24)**.
Technology	nftables, Scapy, NFQUEUE, Suricata as a reference on the same pcap, Loki, Grafana, cargo-fuzz, ASan.
Languages	**Rust** (core; Go is a fair alternative). **Python** (tools and tests). **C++** if the target asks for it.
Milestones	**MVP:** firewall-as-code with attack tests. **v1:** ClientHello parser with JA3/JA4, IDS capture and rules. **v2:** inline IPS, tunnel, honeypot report.
Done when	Your IDS and Suricata are compared on the same pcap (alerts found, throughput, commands). Every parser has a fuzz target that ran clean. Policy tests run in CI. The honeypot, if built, ran in isolation and only after you approved the deployment.
Covers	Palo Alto Networks, F5, Cisco, security vendors, SOC teams.

Track D · router-from-scratch: switch, router, NAT and dynamic routing	Go · Rust (version B) · C++
Goal	A software L2/L3 router with NAT, dynamic routing and automation, in network namespaces started from one Makefile.
Components	**Switch and router (4, protocol 21):** learning switch (MAC aging, flooding, VLAN access and trunk), L3 router (LPM trie, ARP, TTL exceeded, ICMP unreachable), static routes and RIPv2. **NAT44 (11):** connection table with per-protocol timeouts, SNAT/masquerade with port allocation, DNAT, ICMP, fragments, hairpin. **BGP speaker (26, protocol 12):** interop with FRR, netlink route install. **EVPN-VXLAN lab (27)**, **telemetry (28)** and **QoS lab (29):** HTB, fq_codel, CAKE, netem, bufferbloat before and after.
Technology	Network namespaces, containerlab, FRR, netlink, Jinja2 or Ansible, tc, ClickHouse or Prometheus.
Languages	**Go** (primary). **Rust** as version B. **C++** for Cisco, Juniper or Arista-style roles.
Milestones	**MVP:** switch and router in namespaces, pps measured. **v1:** NAT44 and RIPv2. **v2:** BGP with FRR, EVPN-VXLAN lab, QoS report.
Done when	Throughput in pps with a bottleneck analysis. FRR peers with your BGP and your routes land in the kernel. Convergence after a link failure is measured. The whole lab starts with one command.
Covers	Cisco, Juniper, Arista, Nokia, carriers.

5. Optional projects (for a specific target)

Option A · isp-provisioning: subscriber backend for an ISP	Go · Rust · C++
Goal	Simulate what an ISP backend does: subscriber login, address assignment and usage accounting.
Components	RADIUS server (RFC 2865, 2866) with subscribers in PostgreSQL and speed plans passed as RADIUS attributes. The DHCP server from item 5. Session accounting into ClickHouse, usage reports, REST for a portal. Optionally Change of Authorization (RFC 5176) and Diameter basics (item 24).
Technology	PostgreSQL, ClickHouse, Docker, Grafana, FreeRADIUS as a test client.
Languages	**Go** (primary). **Rust**, **C++**. **Java** is common in operator OSS/BSS systems, but that is general knowledge, not taken from postings I checked.
Milestones	**MVP:** Access-Request checked against the subscriber database. **v1:** accounting and reports. **v2:** plans and CoA.
Done when	radtest and eapol_test pass. A usage report for a subscriber. A load test: authentications per second.
Covers	ISP and Wi-Fi companies, Ubiquiti (UniFi), operator OSS/BSS teams.

Option B · voip-gateway: SIP registrar with an RTP relay	Go · Node/TS · Rust
Goal	Build a small VoIP system: registration, call setup and media relay.
Components	SIP registrar and proxy (item 18) with digest authentication, user location in Redis and routing rules. RTP relay (item 19) with jitter and loss statistics. Optional WebRTC gateway. REST for management and call detail record (CDR) export.
Technology	Redis, PostgreSQL, SIPp for tests, Wireshark, Docker.
Languages	**Go** (Pion library for WebRTC). **Node/TS** (Ubiquiti UniFi Talk uses Node.js). **Rust**. **C++** (PJSIP).
Milestones	**MVP:** REGISTER and INVITE between two clients. **v1:** RTP relay and statistics. **v2:** WebRTC and CDR.
Done when	A call between two Linphone clients through your server. A SIPp scenario running in CI. A jitter and loss chart.
Covers	Ubiquiti (UniFi Talk), Cisco (Webex), Motorola (telecom).

6. Schedule at a glance

The detailed week-by-week plan, with the weekly protocol cycle, interview questions and what to cut first, is a separate document (*Plan pracy 6 miesięcy*). Dates assume week 1 starts on Monday 9 November 2026.

Month · weeks	Goal	What lands in the repositories	From the ideas list
PHASE 1 · The 6-month path (about 26 weeks × 8–10 h ≈ 210–260 h)			
1 · w1–4 9 Nov – 6 Dec	Workspace and first protocols	protocols-from-scratch repo with a CI template (gofmt, vet, golangci-lint, test -race, 30 s fuzz smoke) and a Makefile. TFTP in Go (w1–2). Packet parser, ping and traceroute in Go (w3–4).	6 (partly), 2
2 · w5–8 7 Dec – 3 Jan	Rust warm-up and a real server	TFTP ported to Rust with fmt, clippy, cargo-fuzz and proptest (w5). TCP + HTTP/1.1 server in Go with metrics and a wrk benchmark (w6–8; w7–8 contain the holidays: plan a light load).	1, 6
3 · w9–13 4 Jan – 7 Feb	DNS and the track decision	DNS in Go: parser, compression, UDP and TCP, cache, dig verification, fuzzing, benchmark (w9–12). Week 13: buffer, track checkpoint, CV and GitHub profile, start applying for junior and trainee roles.	7
4 · w14–18 8 Feb – 14 Mar	dns-xdp (core project 3)	DNS parser ported to Rust, cache, TCP, EDNS, metrics (w14–16). XDP filter and the pps chart with and without it (w17–18).	7, 9
5 · w19–23 15 Mar – 18 Apr	One specialisation track	Track A, B, C or D as flagship (MVP → v1 → v2 as in Section 4). Details week by week are in the separate work plan.	A: 15, 17, 14, 18, 16 B: 2, 13, 9, 8, 12 C: 25, 21, 20, 5 D: 4, 11, 26, 28, 29
6 · w24–26 19 Apr – 9 May	Proof and polish	Fuzzing sweep over every parser (w24), first upstream pull request and a “what went wrong” post (w25), README polish, demos, pinned repositories, buffer (w26).	24
PHASE 2 · Months 7–12 (outline; order depends on your target)			
7–9	edge-proxy (core project 1)	MVP → v1 → v2 with Redis, PostgreSQL, Kubernetes, k6. Start here first for Cloudflare or Akamai.	1 (details)
10–12	netops-platform (core project 2)	Inventory and config versioning → SNMP and Grafana → sFlow → NATS → ClickHouse, BGP monitor, EVPN lab. Start here first for Cisco, Juniper or Arista.	27, 28, 29

7. Which job requirement you practise where

Requirement in postings	Where you practise it	Companies
Go and/or Rust	All projects. Go: protocols 1, 2, 4; project 1. Rust: the TFTP port, project 3, track C	Cloudflare, Ubiquiti
C and C++ for datapath and vendor roles	Track B (C/eBPF), track A (C datapath), optional DPDK and kernel module (ideas 10, 30)	Nokia, Ericsson, Motorola, Cisco
TCP/UDP, DNS, TLS, HTTP, QUIC	Protocols 2, 4, 9, 10; projects 1 and 3	Cloudflare
eBPF / XDP, packet processing	Protocol 6; project 3; track B	Cloudflare, Akamai, F5
BGP, routing, NETCONF, sFlow	Protocols 12, 13, 15, 21; project 2; track D	Cloudflare, Cisco, Arista, Juniper
5G core, GTP-U, PFCP	Protocols 22, 23; track A	Nokia, Ericsson, Samsung R&D
AAA, RADIUS, Diameter, OSS/BSS	Protocols 17, 24; option A	ISPs, Comarch, Ubiquiti

Requirement in postings	Where you practise it	Companies
IDS/IPS, firewalls, TLS inspection, fuzzing	Protocol 9; track C; a fuzz target in every repository	Palo Alto Networks, F5, Cisco
REST and gRPC APIs, distributed systems	Projects 1 and 2	All
PostgreSQL, ClickHouse, Redis	Projects 1, 2 and option A	Cloudflare, Ubiquiti
Docker, Kubernetes, CI/CD	Projects 1 and 2; the CI template from week 1	Akamai, Cloudflare, Ubiquiti
Prometheus, Grafana, observability	All projects; track B (eBPF observability)	Cloudflare, Akamai, Cisco
OAuth, OWASP, API security	Project 1 (JWT, OAuth2)	Ubiquiti, Cisco
SIP, RTP, WebRTC	Protocols 18 and 19; option B	Ubiquiti, Cisco, Motorola
On-call, owning what you build	Runbook and alerts in projects 1 and 2	Cloudflare, Cisco

8. Checklist for every repository

- **README:** what it is and why, an architecture diagram, one-command start, what is implemented and what is **not**, how it was tested, benchmark results with numbers and the exact command, and a “limitations” section.
- **CI on GitHub Actions:** format, lint, test and build with a green badge. Go: gofmt, go vet, golangci-lint, go test -race. Rust: cargo fmt, clippy with -D warnings, cargo test. C/C++: sanitizers.
- **Tests:** unit tests, at least one test on a real captured packet (small .pcap under testdata/), and integration tests. A fuzz target for every parser, run for at least 30 s without a crash (*go test -fuzz*, *cargo-fuzz*, libFuzzer).
- **Verification with a real tool:** name it in the README (dig, curl, tcpdump, tshark, radtest, SIPp, vtysh, iperf3, Open5GS). Network tests stay on localhost, containers or containerlab.
- **Reproducible lab:** namespaces, containerlab or Docker, so a reader can repeat the result in five minutes. Keep sample pcaps and Mermaid sequence diagrams in the repository.
- **Repository layout:** three core repositories (edge-proxy, netops-platform, dns-xdp), one track repository (mini-5g-user-plane, packet-fastpath, security-gateway or router-from-scratch), and the shared *protocols-from-scratch* with go/ and rust/ folders, where each protocol has its own README.
- **Commit history:** small, imperative, one logical change per commit (*dns: handle compressed names in questions*). A license (for example MIT) and GitHub topics. A short demo (GIF or asciinema) at the top of the README.
- **Rules that never bend:** no fabricated claims (no “works”, “fast” or “tested” without a run you saw), no cryptography from scratch, every length field is untrusted input, no secrets in the repository, no traffic to third-party hosts.
- **What makes you stand out:** one pull request to an open-source project (FRR, Open5GS, Suricata, libbpf, DPDK) even if small, a bug found by fuzzing, a post on what went wrong and how you diagnosed it, and pinned repositories with a one-line specialisation.
- **Honesty:** say you know a technology only when the project works and is on GitHub. Interviewers ask for details: why this way, how you measured, what went wrong. If you use AI coding tools, understand every line.

Appendix · Where the 30 ideas went

Every idea from the project list has a home, so nothing is lost and nothing becomes a 31st repository.

#	Idea in the list	Where it lives in v2	Note
1	epoll TCP server + mini HTTP	Protocol 2 (Go)	C/epoll variant only if you choose track B; compare with nginx
2	Sniffer and protocol analyser	Protocol 6; track B step 1	Parsers are reused in tracks A, B and C
3	TCP/IP stack in userspace (TUN/TAP)	Stretch after protocols 6 and 10	Rust version; test with curl and iperf3 in namespaces
4	Software switch and router	Track D	Plus RIPv2 (protocol 21)

#	Idea in the list	Where it lives in v2	Note
5	WireGuard-style VPN	Protocol 11 + track C	Handshake and crypto from a library
6	ping, traceroute, port scanner	Protocol 6	Scanner only against localhost or namespaces
7	DNS server and resolver	Protocol 4 + project 3	DoT via crypto/tls or rustls
8	XDP L4 load balancer	Track B flagship	Compare with IPVS and nftables
9	XDP firewall and DDoS protection	Project 3 (v2) + track B	pps chart XDP vs nftables
10	DPDK forwarder	Track B stretch	Needs hugepages; run in a VM with virtio
11	NAT44 with conntrack	Track D	Compare with nftables
12	eBPF network observability	Track B	Drop reasons and RTT histograms
13	Traffic generator	Track B tool	Also used to test projects 1 and 3
14	Mini-UPF	Track A flagship	Protocol 22 + 23
15	5G lab with automation	Track A step 1	Open5GS + UERANSIM, pytest
16	4G/5G signalling analyser	Track A optional	Mermaid diagrams from pcap
17	PFCP client/server	Protocol 23, track A	Mock UPF for tests
18	UE simulator and GTP-U generator	Track A	Integrates with 14 and 15
19	AAA server (RADIUS, Diameter)	Protocols 17 and 24; option A	Tests with radtest
20	Mini IDS/IPS	Track C flagship	Compare with Suricata on the same pcap
21	JA3/JA4 fingerprinting	Protocol 9 + track C	ClientHello only, no decryption
22	Honeypot and analytics	Track C optional	Deployment outside localhost only with your explicit approval
23	IPsec/WireGuard site-to-site lab	Track C or D lab	strongSwan and wireguard from libraries
24	Parser fuzzing	Every repository + weeks 24–25	Upstream bugs and pull requests
25	Firewall-as-code	Track C	YAML → nftables, Scapy tests
26	Mini BGP speaker	Protocol 12; netops-platform, track D	Interop with FRR
27	Leaf-spine EVPN-VXLAN lab	netops-platform lab, track D	Convergence and ECMP tests
28	Telemetry collector	Protocols 14, 15; netops-platform	ClickHouse and Prometheus
29	QoS and bufferbloat lab	Track D or netops-platform lab	tc: HTB, fq_codel, CAKE, netem
30	Linux kernel module	Optional stretch	In a VM only; compare with XDP

Note: the requirements in this plan come from job postings, most of which are now closed (Cloudflare, Ubiquiti, Akamai), and from a project-ideas list whose employer matching is indicative and based on typical team profiles. Treat them as a pattern of expectations, not as current openings, and check live postings before applying. The ISP part is based on general knowledge, not on postings I analysed. Time estimates are planning guesses, not measurements.